



Recursion and Backtracking

WEEK# 5-7
DATE:16-4/09/2025

Agenda

- ▶ What is Recursion ?
- ▶ Why is Recursion Important ? What are the resource requirements for recursion?
- ▶ An Example program with function calls and recursion implementation? How are recursive functions different from normal non-recursive functions?
- ▶ Stack Frame? Activation Records? Dynamic Link?
- ▶ Anatomy of a Recursive Call,

What is recursion?

- ▶ When a function calls itself to make the problem smaller.
- ▶ **A Base Case**
 - ▶ Terminate all recursive calls
 - ▶ Clearly identifiable
- ▶ **A Recurrence**
 - ▶ Progression

When should we use recursion?

Recursion is **made for solving problems that can be broken down into smaller, repetitive problems**. It is especially good for working on things that have many possible branches and are too complex for an iterative approach. One good example of this would be searching through a file system.

How are recursive functions different from normal non-recursive functions?

Applied	Recursion is always applied to functions.	Iteration is applied to iteration statements or "loops".
Stack	The stack is used to store the set of new local variables and parameters each time the function is called.	Does not uses stack.
Overhead	Recursion possesses the overhead of repeated function calls.	No overhead of repeated function call.
Speed	Slow in execution.	Fast in execution.
Size of Code	Recursion reduces the size of the code.	Iteration makes the code longer.

Working of recursion

- ▶ Stack frame – **our activation record**
- ▶ keeps the information about local variables, formal parameters, return address and all information passed to the caller function.

Recursion Examples

```
int sumNumbers(int n) {  
    if (n <= 0) {  
        return 0; // Base case: The sum of no numbers is 0.  
    } else {  
        return n + sumNumbers(n - 1); // Recursively add n and the sum of  
        numbers from 1 to n-1  
    }  
}
```

Calculate n raised to the power p

$n^p = n * n * n \dots p \text{ times}$

$n^p = n * n^{p-1}$

$n^0 = 1$

```
using namespace std;

int power(int n, int p){

    if(p==0){
        return 1;
    }

    int prevPower = power(n, p-1);
    return n*prevPower;
}
```


Factorial Function

Find the factorial of a number n 

$$n! = n * n-1 * n-2 * * 1$$

$$n! = n * (n-1)!$$

$$0! = 1 \quad // \text{base case}$$

```
int factorial(int n) {  
    if (n == 0) {  
        return 1; // Base case: 0! is 1  
    } else {  
        return n * factorial(n - 1);  
    }  
    // Recursively calculate factorial  
}
```

Fibonacci Series

```
int fibonacciRecursive(int n) {  
    if (n <= 0) {  
        return 0;        // Base case: Fibonacci(0) is 0  
    } else if (n == 1) {  
        return 1; // Base case: Fibonacci(1) is 1  
    } else {  
        return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);  
    }  
    // Recursively calculate Fibonacci  
}
```

Given array is sorted or not

```
#include<iostream>
using namespace std;

bool sorted(int arr[], int n){
    if(n==1){
        return true;
    }

    bool restArray = sorted(arr+1, n-1);
    return (arr[0]<arr[1] && restArray);
}
```

Print numbers till n..

```
void printNumbers(int n) {  
    if (n <= 0) {  
        return; // Base case: Stop recursion when n is less than or  
equal to 0  
    } else {  
        printNumbers(n - 1); // Recursively call printNumbers with n-1  
        cout << n << " "; // Print the current number  
    }  
}
```

Tower Of Hanoi

- ▶ **Problem Definition: Tower of Hanoi**
- ▶ The **Tower of Hanoi** is a classic recursive problem:
- ▶ You have **3 rods**:
 - ▶ source (where disks are initially stacked)
 - ▶ auxiliary (a helper rod)
 - ▶ destination (where disks must be moved)
- ▶ You have **n disks** of different sizes, stacked on the source rod in decreasing size (largest at bottom, smallest at top).
- ▶ Goal: Move all disks from source → destination rod.
- ▶ Rules:
 - ▶ Only one disk can be moved at a time.
 - ▶ A larger disk **cannot** be placed on a smaller disk.
 - ▶ Only the **top disk** of any rod can be moved.

Tower Of Hanoi

```
void towerOfHanoi(int n, char source, char auxiliary, char destination) {  
    // Base Case: Only one disk left  
    if (n == 1) {  
        cout << "Move disk 1 from " << source << " to " << destination << endl;  
        return;  
    }  
    // Step 1: Move top (n-1) disks from source → auxiliary (using destination as helper)  
    towerOfHanoi(n - 1, source, destination, auxiliary);  
  
    // Step 2: Move the nth (largest) disk from source → destination  
    cout << "Move disk " << n << " from " << source << " to " << destination << endl;  
    // Step 3: Move the (n-1) disks from auxiliary → destination (using source as helper)  
    towerOfHanoi(n - 1, auxiliary, source, destination);  
}
```

Linked List Traversal

```
void traverse(Node* head)
{
    if (head == NULL)
        return;
    // If head is not NULL, print current node
    // and recur for remaining list
    cout << head->data << " ";
    traverse(head->next);
}
```

```
/* Function to reverse the linked list */
Node* reverse(Node* node)
{
    if (node == NULL)
        return NULL;
    if (node->next == NULL) {
        head = node;
        return node;
    }

    Node* node1 = reverse(node->next);
    node1->next = node;
    node->next = NULL;
    return node;
}
```

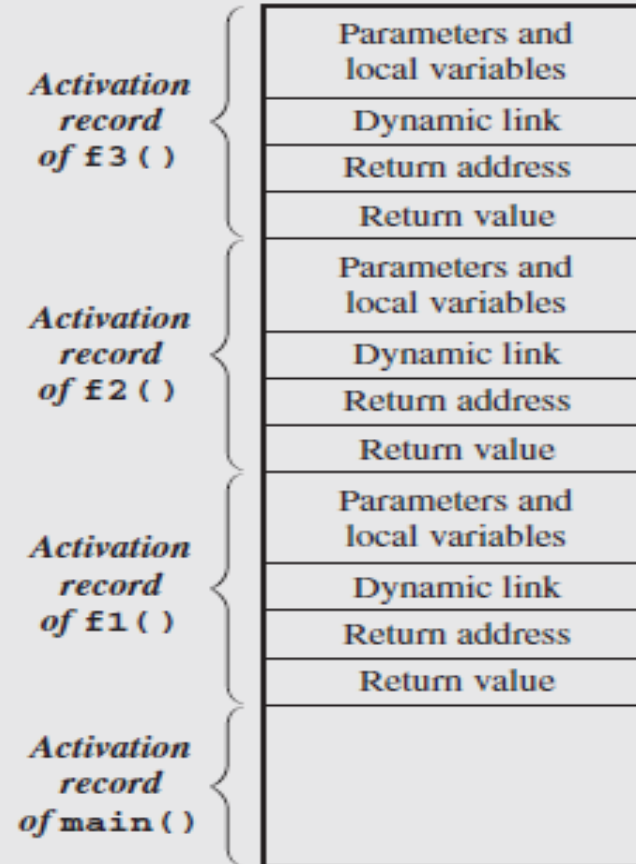
Find the first and last occurrence of an element using recursion

```
int firstocc(int arr[], int n, int i, int key) {
    if (i == n) {
        return -1;
    }
    if (arr[i] == key) {
        return i;
    }
    return firstocc(arr, n, i + 1, key);
}

int lastocc(int arr[], int n, int i, int key) {
    if (i == n) {
        return -1;
    }
    int restArray = lastocc(arr, n, i + 1, key);
    if (restArray != -1) {
        return restArray;
    }
    if (arr[i] == key) {
        return i;
    }
    return -1;
}
```


Stack Frame View

- ▶ Contents of the run-time stack when `main()` calls function `f1()`, `f1()`
- ▶ calls `f2()`, and
- ▶ `f2()` calls `f3()`.



Continue

Stack Frames are an implementation of Activation Records. The dynamic link tells you which activation record to return to when the current function is finished.

Recursive Function

A power function is defined in term of recursive definition and non recursive.

```
1 //Power function
2 #include<iostream>
3 using namespace std;
4 double power (double x, int val){
5     if(val ==0){
6         return 1;
7     }
8     else
9         return x * power (x, val-1);
10 }
11 int main(){
12     cout<<power(5,3)<<endl;
13     return 0;
14 }
```

```
1 //Power function
2 #include<iostream>
3 using namespace std;
4 double non_recursive(double x, int val){
5     for(int result=x;val>=1;--val){
6         result*=x;
7     }
8     return result;
9 }
10
11 int main(){
12     cout<<non_recursive(12,2)<<endl;
13     return 0;
14 }
15
```

Classification of Recursion

- ▶ Tail vs. Non-Tail
- ▶ Direct vs. Indirect
- ▶ Simple vs. Excessive
- ▶ Nested Recursion

Tail vs Non-Tail Recursion

- A function is **tail recursive** if the **recursive call is the last operation** in the function (i.e., nothing is left to do after the recursive call returns).

```
#include <stdio.h>

void tailRecursion(int n) {
    if (n == 0)
        return; // base case
    printf("%d ", n); // work
    tailRecursion(n - 1); // recursive call is last statement
}

int main() {
    tailRecursion(5);
    return 0;
}
```

- Recursive call is the last line → tail recursion.
- No extra work after recursion returns.

Tail vs Non-Tail Recursion

- A function is **non-tail recursive** if it does **something after** the recursive call returns.

```
#include <stdio.h>

void nonTailRecursion(int n) {
    if (n == 0)
        return; // base case
    nonTailRecursion(n - 1); // recursive call
    printf("%d ", n);      // work done after recursive call
}

int main() {
    nonTailRecursion(5);
    return 0;
}
```

- Recursive call is not last (printing happens afterward).
- Must wait for recursive calls to finish before printing.

Tail vs Non-Tail Recursion

Example: Factorial Both Ways

```
int factTail(int n, int res) {  
    if (n == 0) return res;  
    return factTail(n - 1, res * n);  
}
```

```
int factNonTail(int n) {  
    if (n == 0) return 1;  
    return n * factNonTail(n - 1); // multiply happens  
    after return  
}
```

In short:

- **Tail recursion** = last action is recursion (efficient).
- **Non-tail recursion** = extra work after recursion (less efficient).

Direct Recursion vs Indirect Recursion

- ▶ **Direct recursion:** When function calls itself, it is called direct recursion.
- ▶ **Indirect recursion:** When function calls another function and that function calls the calling function, then this is called indirect recursion. For example: function A calls function B and Function B calls function A.

```
69  #include <iostream>
70  using namespace std;
71  //Factorial function
72  int f(int n){
73      if (n <= 1)
74          return 1;
75      else
76          return n*f(n-1);
77  }
78  int main(){
79      int num;
80      cout<<"Enter a number: ";
81      cin>>num;
82      cout<<"Factorial of entered number: "<<f(num);
83      return 0;
84  }
```

```
86  #include <iostream>
87  using namespace std;
88  int fa(int);
89  int fb(int);
90  int fa(int n){
91      if(n<=1)
92          return 1;
93      else
94          return n*fb(n-1);
95  }
96  int fb(int n){
97      if(n<=1)
98          return 1;
99      else
100         return n*fa(n-1);
101  }
102  int main(){
103      int num=5;
104      cout<<fa(num);
105      return 0;
106  }
```


Nested Recursion

- ▶ When a recursive function call is embedded into a recursive function, it is called “Nested Recursion”
- ▶ Example of nested recursion is Ackerman function:
 - ▶ $A(m,n) = n + 1$ if $m == 0$
 - ▶ $A(m,n) = A(m-1, 1)$ if $m > 0 \ \&\& \ n == 0$
 - ▶ $A(m,n) = A(m-1, A(m, n-1))$ if $m > 0 \ \&\& \ n > 0$
- ▶ It is interesting for computational reasons due to the depth of recursion and due to the nested recursion.
- ▶ For example, $A(4, 2)$ is an integer of **19,729** decimal digits

Nested Recursion - Example

```
#include <stdio.h>

int nestedRec(int n) {
    if (n > 10)
        return n - 1;
    else
        return nestedRec(nestedRec(n + 2));
}

int main() {
    printf("%d", nestedRec(5));
    return 0;
}
```

Characteristics of Nested Recursion

1. **Slower than normal recursion** → because for each call it must evaluate another recursive call.
2. **Stack usage is high** → many pending calls remain.
3. **Harder to trace/dry run** compared to tail or non-tail recursion.
4. Used in **special recursive problems** (like McCarthy 91, Ackermann's function).

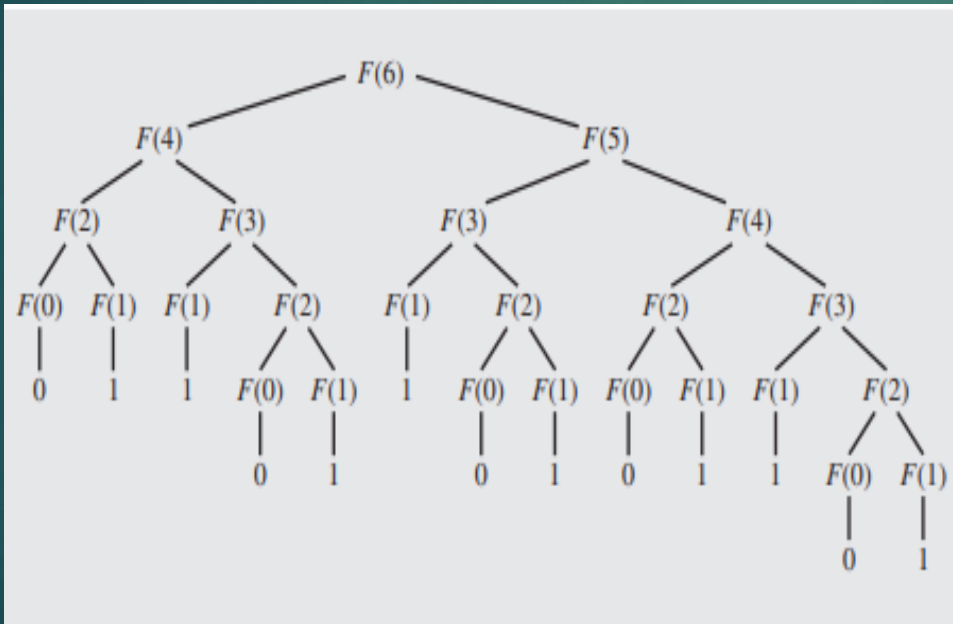
Excessive Recursion

- ▶ Some recursive methods repeats the computations for some parameters, which results in long computation time even for simple cases.
- ▶ This can be implemented in c++ as follows:

```
108 int fib(int n) {  
109     if (n<2) return n;  
110     else  
111         return fib(n-2)+Fib(n-1);  
112 }
```

Excessive Recursion

To show how much this formula is inefficient, let us try to see how $\text{Fib}(6)$ is evaluated.



```
108 int fib(int n) {  
109     if (n<2) return n;  
110     else  
111         return fib(n-2)+Fib(n-1);  
112 }
```

Excessive Recursion: Example

- ▶ For calculating `fib(6)`, the method is called 25 times.
- ▶ The same calculation are repeated again and again because the system forgets what have been already calculated.
- ▶ The method is called $2 \cdot \text{fib}(n+1) - 1$ times to compute `fib(n)`
- ▶ 3,000,000 calls are needed to calculate the 31st element.

Conclusion

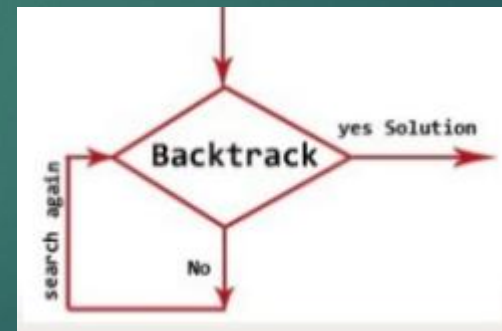
- Recursion is a programming tool. Like any other topic in data structures, it should be used with good judgment.
- Recursive solutions are simplified one for understanding, short and precise code, readability and maintainability are increases.
- Sometimes, slow and hard on resources. Specifically available on only compilers that support recursion.
- Sometimes recursion is faster than iterative approach. Running a simple routine implemented recursively and iteratively and comparing the two run times can help to decide if recursion is advisable.

Example to understand Backtracking

- ▶ A rat is placed at the top-left corner of a maze (`maze[0][0]`).
- ▶ The maze is an $N \times N$ grid.
- ▶ 1 means **open path**, 0 means **blocked cell**.
- ▶ The rat can only **move right or down** (simplest version).
- ▶ Goal: reach the bottom-right corner (`maze[N-1][N-1]`).
- ▶ We'll solve it using **backtracking recursion**.

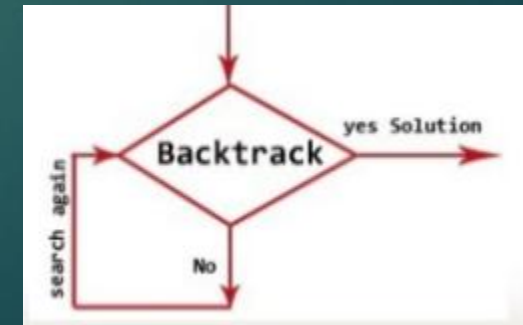
Back Tracking

- Backtracking is an algorithmic technique for solving problems recursively by trying to build a solution incrementally, one piece at a time, removing those solutions that fail to satisfy the constraints of the problem at any point of time.

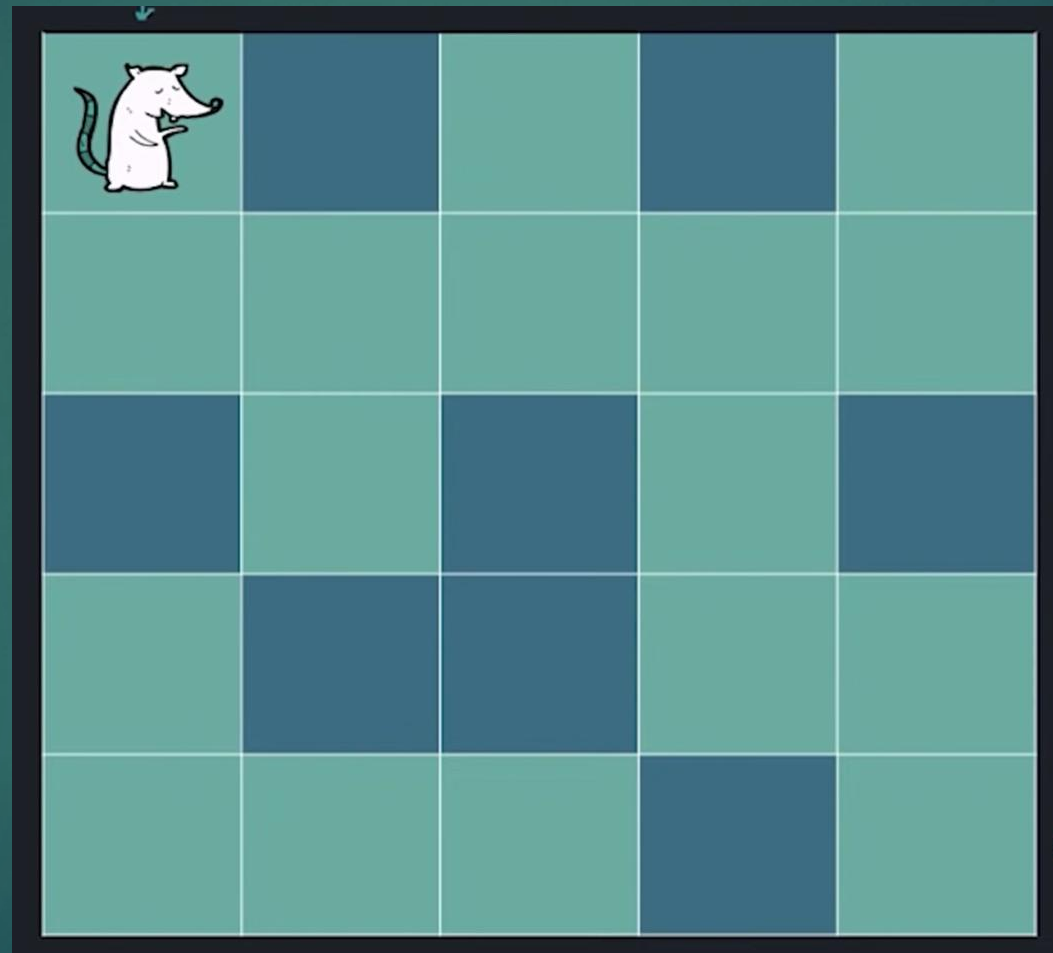


Back Tracking

- ▶ **Key Idea of Backtracking**
- ▶ Start from the **first cell** (0,0).
- ▶ At each step, check:
 - ▶ **Is this cell valid?** (inside maze, not blocked).
 - ▶ If valid, **include it in the path** (mark 1 in solution matrix).
 - ▶ Then try **moving forward** (right, down).
 - ▶ If moving forward fails → **backtrack** (unmark this cell as 0) and try another path.
- ▶ Repeat until reaching the **destination** (N-1, N-1).



Rat in a maze



Walkthrough on Example Maze

Maze:

- ▶ 1 0 0 0
- ▶ 1 1 0 1
- ▶ 0 1 0 0
- ▶ 1 1 1 1

Solution Path (what the program finds):

- ▶ 1 0 0 0
- ▶ 1 1 0 0
- ▶ 0 1 0 0
- ▶ 0 1 1 1

Walkthrough on Example Maze

Step-by-Step Execution

1. Start at $(0, 0) \rightarrow$ it's open (1).
 1. Mark `solution[0][0] = 1`.
2. Try to go **right to $(0, 1)$** $\rightarrow$ blocked (0).
 1. **✗** Not valid, so backtrack.
3. Try to go **down to $(1, 0)$** $\rightarrow$ open (1).
 1. Mark `solution[1][0] = 1`.
4. At $(1, 0)$, try **right to $(1, 1)$** $\rightarrow$ open (1).
 1. Mark `solution[1][1] = 1`.
5. At $(1, 1)$, try **right to $(1, 2)$** $\rightarrow$ blocked. **✗** so backtrack
6. Try **down to $(2, 1)$** $\rightarrow$ open (1).
 1. Mark `solution[2][1] = 1`.
7. At $(2, 1)$, try **right to $(2, 2)$** $\rightarrow$ blocked. **✗** so backtrack
8. Try **down to $(3, 1)$** $\rightarrow$ open (1).
 1. Mark `solution[3][1] = 1`.
9. At $(3, 1)$, try **right to $(3, 2)$** $\rightarrow$ open (1).
 1. Mark `solution[3][2] = 1`.
10. At $(3, 2)$, try **right to $(3, 3)$** $\rightarrow$ open (1).
 1. Mark `solution[3][3] = 1`.
 2. Destination reached!

Walkthrough on Example Maze

Why Backtracking is Needed

- If the rat reaches a **dead end** (blocked cell or outside maze), it **undoes the last move** (backtracks) and tries another direction.
- This prevents getting stuck and ensures all possible paths are explored until a solution is found.

Analogy

Think of the rat as **exploring a maze**:

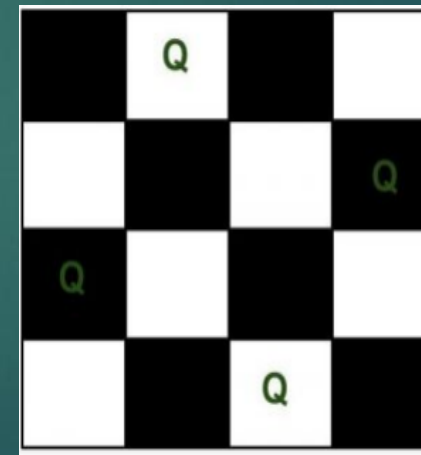
- It marks its path with breadcrumbs ($sol[x][y] = 1$).
- If it hits a wall, it goes back (removes breadcrumb = 0) and tries another route.
- Eventually, it finds the correct path to the exit.

```
bool isSafe(int **arr, int x, int y, int n) {
    if (x < n && y < n && arr[x][y] == 1) {
        return true;
    }
    return false;
}

bool ratinMaze(int** arr, int x, int y, int n, int** solArr) {
    if ((x == (n - 1)) && (y == (n - 1))) {
        solArr[x][y] = 1;
        return true;
    }
    if (isSafe(arr, x, y, n)) {
        solArr[x][y] = 1;
        if (ratinMaze(arr, x + 1, y, n, solArr)) {
            return true;
        }
        if (ratinMaze(arr, x, y + 1, n, solArr)) {
            return true;
        }
        solArr[x][y] = 0; //backtracking
        return false;
    }
    return false;
}
```

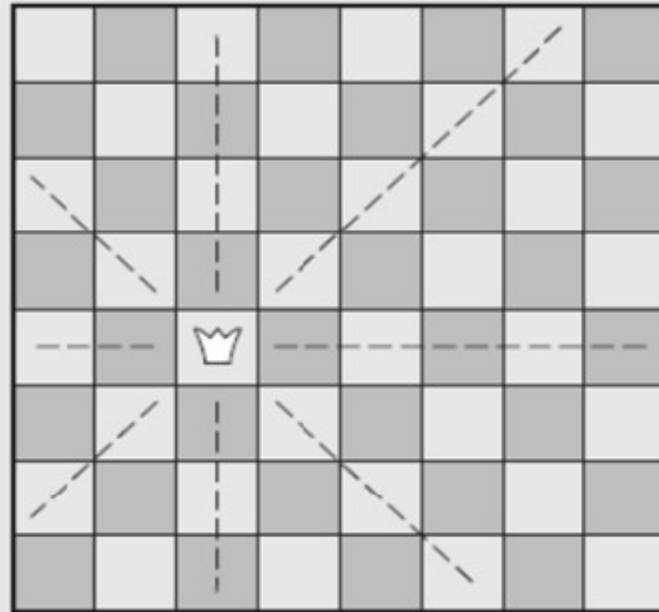
N-Queen Problem

- We have **N queens** and **N x N chess board**. The objective of this problem is such that we need to place all N queens on N x N chess board in such a manner that **no two queens** are under attack to each other.
- **Legal move** attacking means no 2 queens in same row, col, and diagonal
- Two queens will be under attack if one of the following conditions is true:-
 - firstly, if they are in the same row.
 - secondly, if they are in the same column.
 - finally, if they are in the same diagonal.

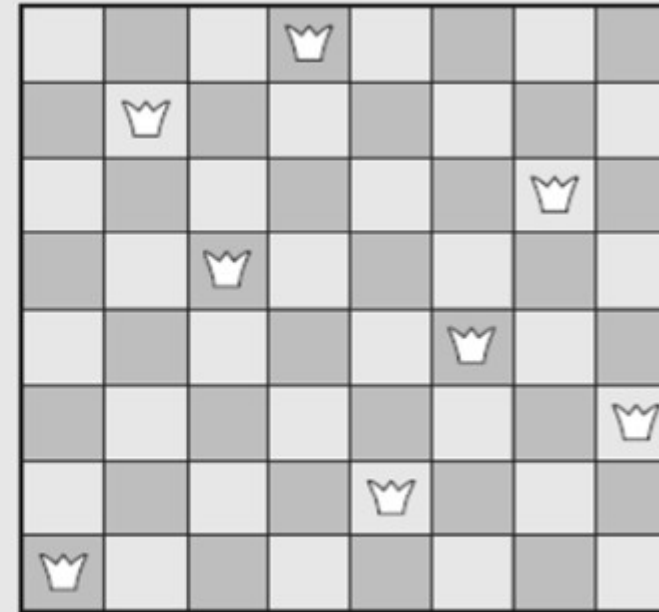


N-Queen Problem

-



(a)



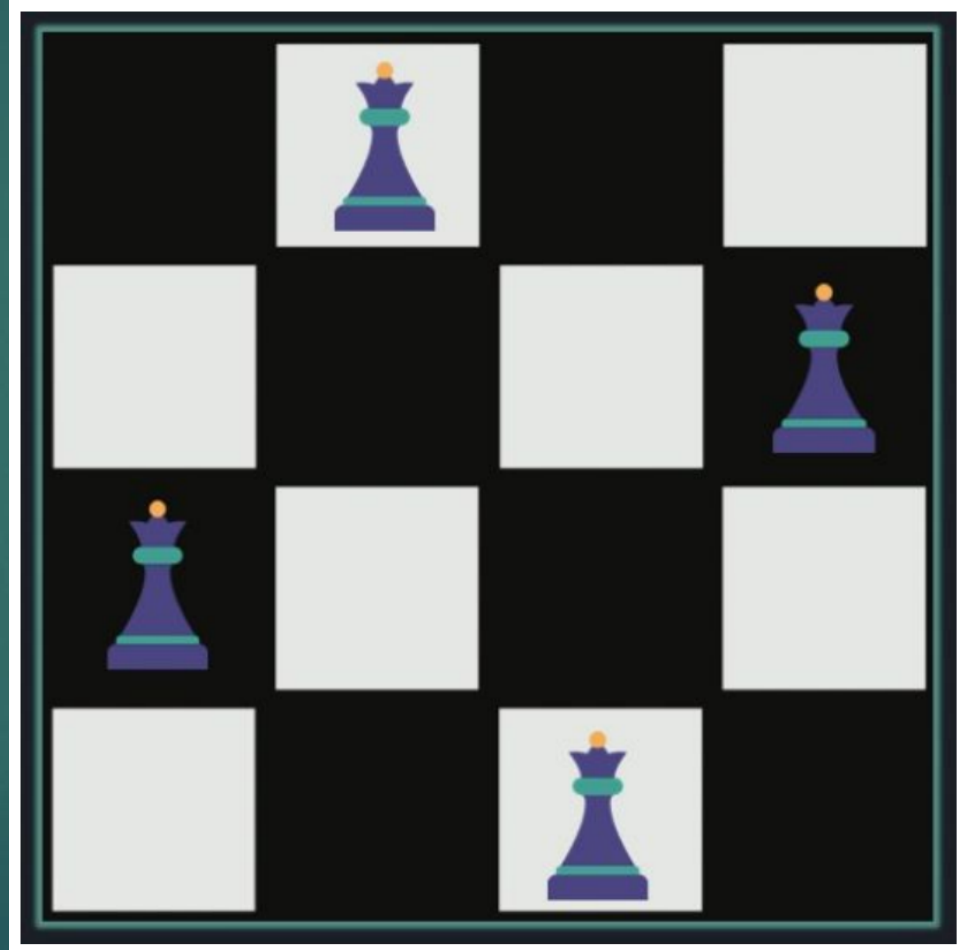
(b)

Backtracking Algorithm

- The idea is to place **queens one by one** in **different columns, starting from the leftmost column**. When we place a queen in a column, we check for **clashes** with already **placed queens**.
- In the current column, if we find a row for which there is **no clash**, we mark this row and column as part of the **solution**. If we do not find such a row due to clashes then we **backtrack** and **return false**.

Backtracking Algorithm

1. Start in the leftmost column
2. If all queens are placed
→ return true
3. Try all rows in the current column. Do following for every tried row.
 - a. If the queen can be placed safely in this row then mark this [row, column] as part of the solution and recursively check if placing queen here leads to a solution.
 - b. If placing the queen in [row, column] leads to a solution then return true.
 - c. If placing queen doesn't lead to a solution then unmark this [row, column] (Backtrack) and go to step (a) to try other rows.
4. If all rows have been tried and nothing worked, return false to trigger backtracking.



```
bool isSafe(int** arr, int x, int y, int n) {
    for (int row = 0; row < x; row++) {
        if (arr[row][y] == 1) {
            return false;
        }
    }
    int row = x;
    int col = y;
    while (row >= 0 && col >= 0) {
        if (arr[row][col] == 1) {
            return false;
        }
        row--;
        col--;
    }
    row = x;
    col = y;
    while (row >= 0 && col < n) {
        if (arr[row][col] == 1) {
            return false;
        }
        row--;
        col++;
    }
    return true;
}
```

```
bool nQueen(int** arr, int x, int n) {
    if (x >= n) {
        return true;
    }
    for (int col = 0; col < n; col++) {
        if (isSafe(arr, x, col, n)) {
            arr[x][col] = 1;

            if (nQueen(arr, x + 1, n)) {
                return true;
            }

            arr[x][col] = 0; //backtracking
        }
    }
    return false;
}
```



Step-by-Step Execution:

- ▶ **Row 0, try col 0**
 - ▶ Attempt (0,0) → **OK** (no queens yet).
 - ▶ **Place** queen at (0,0).
- ▶ **Row 1, try col 0**
 - ▶ Attempt (1,0) → **Rejected**: same **column** as queen at (0,0).
- ▶ **Row 1, try col 1**
 - ▶ Attempt (1,1) → **Rejected**: same \ **diagonal** as queen at (0,0) (since $1-1 = 0$, $0-0 = 0$).
- ▶ **Row 1, try col 2**
 - ▶ Attempt (1,2) → **OK**.
 - ▶ **Place** queen at (1,2).
- ▶ **Row 2, try col 0**
 - ▶ Attempt (2,0) → **Rejected**: same **column** as (0,0).
- ▶ **Row 2, try col 1**
 - ▶ Attempt (2,1) → **Rejected**: same / **diagonal** as (1,2) (because $1+2 = 3$ and $2+1 = 3$).
- ▶ **Row 2, try col 2**
 - ▶ Attempt (2,2) → **Rejected**: same \ **diagonal** as (0,0) ($2-2 = 0$ equals $0-0$).
- ▶ **Row 2, try col 3**
 - ▶ Attempt (2,3) → **Rejected**: same \ **diagonal** as (1,2) ($2-3 = -1$, $1-2 = -1$).
- ▶ **Row 2, try col 4**
 - ▶ Attempt (2,4) → **OK**.
 - ▶ **Place** queen at (2,4).

► **Row 2, try col 0**

- Attempt (2,0) → **Rejected**: same **column** as (0,0).

► **Row 2, try col 1**

- Attempt (2,1) → **Rejected**: same / **diagonal** as (1,2) (because $1+2 = 3$ and $2+1 = 3$).

► **Row 2, try col 2**

- Attempt (2,2) → **Rejected**: same \ **diagonal** as (0,0) ($2-2 = 0$ equals $0-0$).

► **Row 2, try col 3**

- Attempt (2,3) → **Rejected**: same \ **diagonal** as (1,2) ($2-3 = -1$, $1-2 = -1$).

► **Row 2, try col 4**

- Attempt (2,4) → **OK**.
- **Place** queen at (2,4).

► **Row 3, try col 0**

- Attempt (3,0) → **Rejected**: same **column** as (0,0).

► **Row 3, try col 1**

- Attempt (3,1) → **OK**.
- **Place** queen at (3,1).

► **Row 4, try col 0**

► Attempt (4,0) → **Rejected**: same **column** as (0,0).

► **Row 4, try col 1**

► Attempt (4,1) → **Rejected**: same **column** as (3,1).

► **Row 4, try col 2**

► Attempt (4,2) → **Rejected**: same **column** as (1,2).

► **Row 4, try col 3**

► Attempt (4,3) → **OK**.

► **Place** queen at (4,3).

This placement sequence corresponds to the solution (columns per row):

[0, 2, 4, 1, 3]

That is:

- Row 0 → col 0
- Row 1 → col 2
- Row 2 → col 4
- Row 3 → col 1
- Row 4 → col 3